4.2 Induced Distributions after Fine-Tuning

Now, with the above preparation, we can show that the induced distribution derived by this soft optimal policy is actually equal to our target distribution.

Theorem 1 (Theorem 1 in Uehara et al. (2024))**.** *Let $p^\star(\cdot)$ be an induced distribution at time 0 from optimal policies $\{p_t^\star\}_{t=T+1}^1$, i.e., $p^\star(x_0) = \int \{\prod_{t=T+1}^1 p_t^\star(x_{t-1}|x_t)\} dx_{1:T}$. The distribution $p^\star$ is equal to the target distribution (10), i.e.,*

$$p^\star(x) = p_r(x).$$

This theorem states that after solving (18) and obtaining soft optimal policies, we can sample from the target distribution by sequentially running policies from $p_{T+1}^\star$ to $p_1^\star$. Thus, (18) serves as a natural objective for fine-tuning. This fact is also useful in deriving a connection with classifier guidance in Section 8.

Marginal distributions. We can derive the marginal distribution as follows.

Theorem 2 (Theorem 2 in Uehara et al. (2024))**.** *Let $p_t^\star(x_t)$ be the marginal distributions at t induced by soft-optimal policies $\{p_t^\star\}_{t=T+1}^1$, i.e., $p_t^\star(x_t) = \int \{\prod_{k=T+1}^{t+1} p_k^\star(x_{k-1}|x_k)\} dx_{t+1:T}$. Then,*

$$p_t^\star(x_t) = \frac{\exp(v_t(x_t)/\alpha) p_t^{\mathrm{pre}}(x_t)}{C},$$

where $v_t(\cdot)$ is the soft-value function.

Interestingly, the normalizing constant is independent of t in the above theorem.

Posterior distributions. We can derive the posterior distribution as follows.

Theorem 3 (Theorem 3 in Uehara et al. (2024))**.** *Denote the posterior distribution of x_t given x_{t-1} for the distribution induced by soft-optimal policies $\{p_t^\star\}_{t=T+1}^1$ by $\{p^\star\}_t^b(\cdot \mid \cdot)$. We define the analogous objective for a pre-trained policy and denote it by $p_t^{\mathrm{pre}}(\cdot \mid \cdot)$. Then, we get*

$$\{p^\star\}_t^b(x_t|x_{t-1}) = p_t^{\mathrm{pre}}(x_t|x_{t-1}).$$

This theorem indicates that after solving (18), the posterior distribution induced by pre-trained models remains preserved. This property plays an important role in constructing PCL (path consistency learning) in Section 6.3.

Remark 4 (Continuous-time formulation)**.** *For simplicity, our explanation is generally based on the discrete-time formulation. However, as training of diffusion models could be formulated in the continuous-time formulation (Song et al., 2021), we can still extend most of our discussion of fine-tuning in our tutorial in the continuous-time formulation. For example, the above Theorems are extended to the continuous time formulation in Uehara et al. (2024).*

Table 1: Description of each RL algorithm for fine-tuning diffusion models (note that value-weighted sampling is technically not a fine-tuning algorithm.) Note (1) "Without learning value functions" refers to the capability of algorithms to directly utilize non-differentiable black-box reward feedback, bypassing the necessity to train differentiable reward functions, (2) "Distribution-constrained" indicates that the algorithms are designed to maintain proximity to pre-trained models. Based on it, the practical recommendation of algorithms is summarized in Figure 3.

	Memory efficiency	Computational efficiency	Without learning value functions	Distribution-constrained
Soft PPO	✓		✓	
Reward backpropagation		✓	✓	
Reward-weighted MLE	✓		✓	✓
Value-weighted sampling	✓			✓
Path consistency learning	✓			✓

5 RL-Based Fine-Tuning Algorithms 1: Non-Distribution-Constrained Approaches

So far, we have explained how to frame fine-tuning diffusion models as the RL problem in entropy-regularized MDPs. Moving forward, we summarize actual algorithms that can solve the RL problem of interest described by Equation (18). In this section, we introduce two algorithms: PPO and direct reward backpropagation.

These algorithms are originally designed to optimize reward functions directly, meaning they operate effectively even without entropy regularization (i.e., $\alpha = 0$). Consequently, they are well-suited for generating samples with high rewards that may not be in the original training dataset. More distribution-constrained algorithms that align closely with pre-trained models will be discussed in the subsequent section (Section 4.2). Therefore, we classify algorithms in this section (i.e., PPO and direct reward backpropagation) as non-distribution-constrained approaches. The whole summary is described in Table 1.

5.1 Soft Proximal Policy Optimization (PPO)

In order to solve Equation (18), Fan et al. (2023); Clark et al. (2023) propose using PPO (Schulman et al., 2017). PPO has been widely used in RL, as well as, in the literature in fine-tuning LLMs, due to its stability and simplicity. In the standard context of RL, this is especially preferred over Q-learning when the action space is high-dimensional.

The PPO algorithm is described in Algorithm 1. This is an iterative procedure of updating a parameter θ. Each iteration comprises two steps: firstly, samples are generated by executing

Algorithm 1 Soft PPO

Require: Pre-trained model $\{\mathcal{N}(\rho(x_t, t; \theta_{\text{pre}}), \sigma^2(t))\}_{t=T+1}^{1}$, batch size m, parameter $\alpha \in \mathbb{R}^+$, learning rate γ

Initialize: $\theta_1 = \theta_{\text{pre}}$

for $s \in [1, \cdots, S]$ **do**

Collect m samples $\{x_t^{(i)}(\theta)\}_{t=T}^{0}$ from a current diffusion model (i.e., generating by sequentially running polices $\{\mathcal{N}(\rho(x_t, t; \theta), \sigma^2(t))\}_{t=T+1}^{1}$ from $t = T+1$ to $t = 1$)

Update as follows (several times if needed):

$$\theta_{s+1} \leftarrow \theta_s - \frac{\gamma}{m} \nabla_\theta \sum_{t=T+1}^{1} \sum_{i=1}^{m} \left[\min \left\{ \tilde{r}_t(x_0^{(i)}, x_t^{(i)}) \frac{p(x_{t-1}^{(i)} | x_t^{(i)}; \theta)}{p(x_{t-1}^{(i)} | x_t^{(i)}; \theta_s)}, \right. \right. \tag{21}$$
$$\left. \left. \tilde{r}_t(x_0^{(i)}, x_t^{(i)}) \cdot \text{Clip} \left(\frac{p(x_{t-1}^{(i)} | x_t^{(i)}; \theta)}{p(x_{t-1}^{(i)} | x_t^{(i)}; \theta_s)}, 1 - \epsilon, 1 + \epsilon \right) \right\} \right] |_{\theta = \theta_s},$$

where

$$\tilde{r}_t(x_0, x_t) := -r(x_0) + \underbrace{\alpha \frac{\|\rho(x_t, t; \theta) - \rho(x_t, t; \theta^{\text{pre}})\|^2}{2\sigma^2(t)}}_{\text{KL term}}.$$

end for

Output: Policy $\{p_t(\cdot \mid \cdot; \theta_S)\}_{t=T+1}^{1}$

current policies to construct the loss function (inspired by policy gradient formulation); secondly, the parameter θ is updated by computing the gradient of the loss function.

PPO offers several advantages. The approach is known for its stability and relatively straightforward implementation. Stability comes from the conservative parameter updates. Indeed, PPO builds upon TRPO (Schulman et al., 2015), where parameters are conservatively updated with a KL penalty term (between θ_{s+1} and θ_s) to prevent significant deviation from the current parameter. This gives us stability in the optimization landscape. Furthermore, in Algorithm 1, we do not necessarily need to rely on value functions, although they could be useful for variance reduction. As discussed in the subsequent subsection, this can be advantageous compared to other methods, especially since learning value functions can be challenging in high-dimensional spaces, particularly within the context of diffusion models.

5.2 Direct Reward Backpropagation

Another standard approach is a differentiable optimization (Clark et al., 2023; Prabhudesai et al., 2023; Uehara et al., 2024), where gradients are directly propagated from reward functions to update policies.

The entire algorithm is detailed in Algorithm 2. This reward backpropagation entails an iterative process of updating a parameter θ. Each iteration comprises two steps; firstly, samples are generated by executing current policies to approximate the expectation in the loss function, which is directly derived from (18); second, the current parameter θ is updated by computing the gradient of the loss

1. Learning $q_y(x,t)$ through regression by gathering data containing $\{x_t, y\}$. Here, for each (x_0, y), using a forward policy for the pre-training (from 0 to T), we obtain x_t starting from x_0.

2. During inference, sequentially execute $\{p_t^{\text{gui}}(\cdot|x_t,y)\}_{t=T+1}^1$ by leverage the fact that compared to the policy in pre-trained diffusion models, the mean of $\{p_t^{\text{gui}}(\cdot|x_t,y)\}_{t=T+1}^1$ is just shifted with an additional term $\nabla_x \log q_y(x,t)$.

Notably, the method described above can be reduced to value-weighted sampling (cf. Section 6.2), when $r(x,y) = \log p(y|x)$. Inspired by such an observation, we apply an RL-based fine-tuning method for conditional generation as below.

8.2 RL-Based Fine-Tuning for Conditional Generation

In this section, following Zhao et al. (2024), we explain an RL-based fine-tuning approach for conditional generation. The core idea is that setting $r(x,y) = \log p(y|x)$ allows us to sample from the target conditional distribution (37), drawing on the insights discussed in Section 4.2.

More precisely, introducing a distribution over y as $q(\cdot)$, we formulate the following RL problem:

$$\{p_t^\star\}_t = \underset{\{p_t \in [(\mathcal{Y},\mathcal{X}) \to \Delta(\mathcal{X})]\}_{t=T+1}^1}{\operatorname{argmax}} \mathbb{E}_{\{p_t(\cdot|\cdot,y)\},y\sim\Pi}[\log p(y|x_0) - \Sigma_{t=T+1}^1 \mathrm{KL}(p_t(\cdot|x_t,y)\|p_t^{\text{pre}}(\cdot|x_t))]. \tag{38}$$

Compared to the objective function (18) in Section 4.2, the main differences are: (1) policy space is expanded to accommodate additional control (i.e., y): $p_t(\cdot|\cdot,\cdot) \subset [\mathbb{R}^d \times \mathcal{Y} \to \Delta(\mathbb{R}^d)]$, (2) the fine-tuning objectives are set as log-likelihoods: $\log p(y|x)$. Then, following Theorem 1, we can establish the following result.

Corollary 2. *The distribution induced by* $\{p_t^\star\}_{t=T+1}^1$ *(i.e.,* $\int \{\prod_{t=T+1}^1 p_t^\star(x_{t-1} \mid x_t, y)\} dx_{1:T}$ *is the target conditional distribution* (37)*.*

The above corollary suggests that for conditional generation, we can utilize various off-the-shelf algorithms discussed in Section 4.2 and 6, such as PPO, reward backpropagation, and reward-weighted MLE, as well as value-weighted sampling.

9 Connection with Flow-Based Diffusion Models

We elucidate the close relationship between bridge matching and fine-tuning, as summarized in Table 2. Bridge (flow) matching (Liu et al., 2022; Tong et al., 2023; Lipman et al., 2023; Liu et al., 2022; Shi et al., 2024; Albergo and Vanden-Eijnden, 2022), has recently gained popularity as a distinct type of diffusion model from score-matching-based ones. These works suggest that training and inference speed will be accelerated due to the ability to define a more direct trajectory from noise to data distribution than score-matching-based diffusion models.

The connection between bridge matching and fine-tuning becomes evident in the continuous-time formulation. To begin, we first provide a brief overview of the continuous-time formulation of score-matching-based diffusion models.

Table 2: Comparison between training score/flow-based diffusion models and fine-tuning. We optimize the parameter θ in $\mathbb{P}^\theta$, which represents the induced distribution over trajectories associated with θ.

	Loss	Note of $\mathbb{Q}$
Score-based training	$\mathrm{KL}(\mathbb{Q}^{\mathrm{time}} \| \mathbb{P}^\theta)$	$\mathbb{Q}^{\mathrm{time}}$: Time-reversal of reference SDE (reference SDE is defined from 0 to T where the initial distribute at 0 is data distribution)
Flow-based training	$\mathrm{KL}(\mathbb{Q}^{\mathrm{doob}} \| \mathbb{P}^\theta)$	$\mathbb{Q}^{\mathrm{doob}}$: Coupling between the reference SDE from T to 0 conditioning at 0 (dist. over $x_{T:1}\|x_0$) and a data distribution at time 0 (dis. over x_0). Note that reference SDE is defined from T to 0.
Fine-tuning	$\mathrm{KL}(\mathbb{P}^\theta \| \mathbb{Q}^{\mathrm{fine}})$	$\mathbb{Q}^{\mathrm{fine}}$: Coupling between the pre-trained SDE from T to 0 conditioned on 0 (dist. over $x_{T:1}\|x_0$) and a data distribution at time 0 (dis. over x_0)

9.1 Continuous Time Formulation of Score-Based diffusion models

We provide a brief overview of the continuous-time formulation of diffusion models (Song et al., 2021), further expanding Section 1.1.1. In this formulation, we initially define the forward reference SDE, spanning from time 0 to T (e.g., variance exploding (VE) process or variance preserving (VP) process), by setting an initial distribution at 0 as the data distribution. Additionally, the reference SDE is tailored to converge to an easy distribution at time T, such as the normal distribution. Subsequently, we consider the time-reversal SDE (Anderson, 1982), progressing from T to 0. If we could learn this time-reversal SDE, it would enable us to sample from the data distribution by starting with the easy initial distribution (at T) and following the time-reversal SDE from T to 0 at inference time.

Training. Now, the remaining question is how to learn this time-reversal SDE. Here, by representing the induced distribution over trajectories from T to 0 with this time-reversal SDE as $\mathbb{Q}^{\mathrm{time}}$, our goal is to train a new SDE, parameterized by θ in the neural network, such that the induced distribution $\mathbb{P}^\theta$ over trajectories associated with θ aligns with $\mathbb{Q}^{\mathrm{time}}$. Specifically, the objective is to minimize the following losses:

$$\operatorname*{argmin}_\theta \mathrm{KL}(\mathbb{Q}^{\mathrm{time}} \| \mathbb{P}^\theta). \tag{39}$$

With some algebra, leveraging the observation that the time-reversal SDE comprises the original forward SDE and the scoring term of the *marginal* distribution (Anderson, 1982) as in (2), Song et al. (2021) demonstrates that this KL loss (39) is approximated as

$$\hat{\theta}_{\mathrm{pre}} = \operatorname*{argmin}_\theta \mathbb{E}_{t\sim\mathrm{Uni}([0,T]),x_t\sim q_{t|0}(x_t|x_0),x_0\sim p^{\mathrm{pre}}}[\{\sigma_t^\diamond\}^2 \|\nabla_{x_t} \log q_{t|0}(x_t \mid x_0) - s_\theta(x_t,t)\|^2], \tag{40}$$

which is equal to (4) where the weight $\lambda(t)$ is $\{\sigma_t^\diamond\}^2$ (for notation, refer to Section 1.1.1).

9.2 Flow-Based (Bridge-Based) Diffusion Models

We adopt the explanation outlined in Liu et al. (2022). In this framework, we begin by considering a reference SDE (from T to 0), $x_t = z_{T-t}$ where z_t is defined as Brownian motion:

$$dz_t = dw_t, \quad t \in [0, T]. \tag{41}$$

Subsequently, we consider the coupling of the reference SDE conditioned on state at time 0 (i.e., distribution of $x_{0:T}$ conditional on x_0) and the data distribution at time 0, denoting the resulting distribution as $\mathbb{Q}^{\text{doob}}$. Informally, this $\mathbb{Q}^{\text{doob}}$ is characterized as follows.

Informal characterization of $\mathbb{Q}^{\text{doob}}$. When discretizing the time step, by denoting the distribution induced by the reference SDE as $p^{\text{ref}}(x_T, \cdots, x_0)$, we define

$$\mathbb{Q}^{\text{doob},T}(x_T, \cdots, x_0) := p^{\text{ref}}(x_T, \cdots, x_1 \mid x_0) p^{\text{pre}}(x_0).$$

The distribution $\mathbb{Q}^{\text{doob}}$ above is a limiting distribution obtained by making the discretization step small.

Training. In bridge-based diffusion models, our objective is to train a new SDE, parametrized by θ (e.g., a neural network), to align the induced distribution $\mathbb{P}^\theta$ with $\mathbb{Q}^{\text{doob}}$. Specifically, bridge-based training aims to minimize

$$\operatorname*{argmin}_\theta \mathrm{KL}(\mathbb{Q}^{\text{doob}} \| \mathbb{P}^\theta). \tag{42}$$

While the method to minimize the above is currently unclear, we can derive a trainable objective through algebraic manipulation. Initially, we observe that the reference SDE (41), conditioned on state x_0 at time 0 ($z_T = b$), is expressed as:

$$t \in [0, T]; dz_t^b = g^b(z_t^b, t)dt + dw_t, \quad g^b(z_t^b, t) = \frac{b - z_t^b}{T - t}, \tag{43}$$

using the seminal "Doob's h-transform" (Rogers and Williams, 2000). This SDE is commonly known as the Brownian bridge. Now, we consider a new SDE parameterized by θ:

$$t \in [0, T]; dz_t = s(z_t, t; \theta)dt + dw_t. \tag{44}$$

Then, we can demonstrate that minimizing the KL loss in (42) is equivalent to

$$\operatorname*{argmin}_\theta \mathbb{E}_{t \sim \mathrm{Uni}[0,T], z_t \sim q_t^{x_0}, x_0 \sim p^{\text{pre}}} \left[\| g^{x_0}(z_t, t) - s(z_t, t; \theta) \|_2^2 \right] \tag{45}$$

where $q_t^{x_0}$ denotes the induced distribution at time t following SDE (43) conditioned on x_0, and $s : \mathcal{X} \times [0, T] \to \mathcal{X}$ is the model we aim to optimize.